Git: Delete and Recover Commit & Branch - Step-by-Step Guide

1. Delete a Commit (Local)

- A. Delete latest commit and changes:

  git reset --hard HEAD~1

- B. Delete but keep changes in working directory:

  git reset --mixed HEAD~1

- C. Delete but keep changes staged:

  git reset --soft HEAD~1

- D. Delete any commit interactively:

  git rebase -i HEAD~n (then mark commit as 'drop')

- E. Revert a commit safely (non-destructive):

  git revert <commit-hash>

2. Recover a Deleted Commit

- A. View recent history:

  git reflog

Restore at the same branc
git reset --hard f6g7h8i

- B. Find the commit hash from reflog

- C. Recover it:

recover by creating a new branch named recovered

  git checkout -b recovered <commit-hash>

- D. Or apply it elsewhere:

  git cherry-pick <commit-hash>

3. Delete a Branch

- A. Delete a local branch:

  git branch -d branch-name

- B. Force delete a local branch:

  git branch -D branch-name

- C. Delete a remote branch:

  git push origin --delete branch-name

4. Recover a Deleted Branch

- A. Run reflog:

  git reflog

- B. Find the commit hash before deletion
- C. Recreate the branch:

  git checkout -b branch-name <commit-hash>

5. Tips and Warnings

- Use 'git log --oneline --graph' for visual history.
- Avoid 'reset --hard' on shared branches.
- Commits are garbage collected after ~30 days.
- Use 'git fsck --lost-found' to find unreachable commits (advanced).